Automata — DS preparation

Dr-Ing Ines Baccouche · 9 exercises covering every chapter · built from her Tutorial 1 (series.pdf), her slides and the class board · solutions at the end

How to use this in the time you have

Order	Exercise	Covers	Time
1 ★	E5 — (a|b)*abb	Glushkov → NFA → DFA → minimal (3 chapters in 1)	20 min
2 ★	E4 — aba / bab / bbb	explain a language, Glushkov, DFA, complement	20 min
3 ★	E7 — read 4 automata	automaton → language, deterministic / complete	15 min
4 ★	E8 — minimise	unreachable state, Step0/Step1..., fusion	15 min
5 ★	E9 — NFA → DFA (+ ε)	subset construction, ε-closure, complement	15 min
6	E3 — at least / at most / even	language → RegEx + DFA	10 min
7	E2 — true or false	RegEx equalities, counter-examples	10 min
8	E6 — table → automaton	drawing, traces, properties	10 min
9	E1 — numbers	RegEx notation \d, {n}	5 min

★ = do these first: they are the long DS-style questions. Work on paper, in her format (tables, Step0..., δ'([q0],a) = ...). When stuck, open the cheat sheet section given in each exercise, then continue. Check the solution only after you finish the exercise.

Exercises

E1 — Numbers (Ch1, RegEx notation)

Tutorial 1 · Ex 1 Cheat sheet: Ch1 §3 notation

Alphabet {0,...,9}, d = [0-9]. Give the RegEx of:

1. numbers of exactly three digits;
2. numbers that can be divided by 5;
3. numbers that start with 4 and end with 7;
4. numbers of three digits that do not start with 0.

E2 — True or false? (Ch1)

slides 27–29 style Cheat sheet: Ch1 §6 and §7

1. True or false? Justify with a rule (true) or a counter-example word (false):
   `(a*b)* = (a|b)*b` · `(ab)*a = a(ba)*` · `(a|b)* = (a*b*)*` · `(ab)* = a*b*`
2. R = `(a|b)*bb` and S = `b(a|b)*`. Give a word: in R not in S · in S not in R · in both · in neither.

E3 — RegEx and automaton of a language (Ch1 + Ch2)

Tutorial 1 · Ex 4 Cheat sheet: Ch1 §4, Ch3 §3

Over {a,b}, give the RegEx **and** a deterministic automaton of the words containing:

1. at least one a;
2. at most one a;
3. an even number of a.

E4 — Explain and build (Ch1 + Ch2 + Ch3)

★ DS **Tutorial 1 · Ex 2** Cheat sheet: Glushkov, Ch3 §4 (flip)

For each language, explain it in words and draw an automaton that recognises it using a construction method:

1. L denoted by `(aba)|(bab)`;
2. L denoted by `(aba)*|(bab)*`;
3. L = { u ∈ {a,b}* | u contains the factor bbb };
4. L = { u ∈ {a,b}* | u does not contain the factor bbb }. Also give its RegEx.

E5 — The complete chain: (a|b)*abb (Ch2 + Ch3)

★ DS **Tutorial 1 · Ex 3** Cheat sheet: Glushkov, NFA→DFA, Minimisation

1. Build the Glushkov automaton of `(a|b)*abb` (numbering, successors table, F, drawing).
2. Is it deterministic? Justify.
3. Convert it to a DFA (subset construction).
4. Is your DFA minimal? Prove it with the Step method.
5. Trace **aabb** and **abab** on the DFA.

E6 — From a table to an automaton (Ch2)

Tutorial 1 · Ex 5 Cheat sheet: Ch2 §1, §5

	a	b
0	1	0
1	0	2
2	3	0
3	1	2

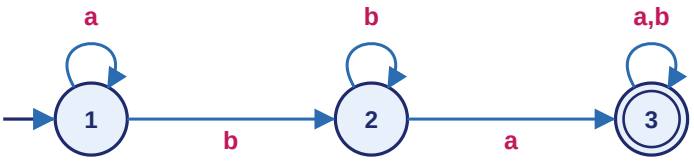
1. Draw the automaton (0 initial, 2 final).
2. Is it complete? Unambiguous? Deterministic?
3. Are **ab**, **aab**, **abab**, **abb** accepted? Write the traces.
4. Is it minimal? (Step method)

E7 — Give the language of each automaton (Ch2 + Ch3)

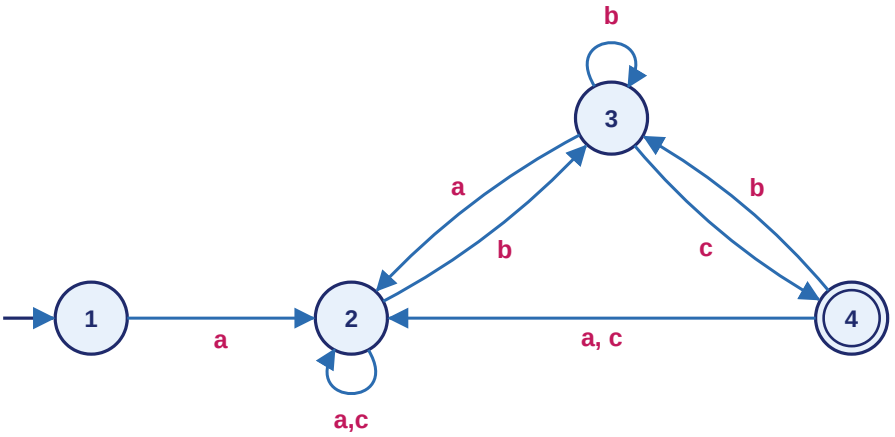
★ DS · Tutorial 1 · Ex 6 · Cheat sheet: Ch3 §1, Extra A (Arden)

For each automaton: describe the language in words, give a RegEx, and say if it is deterministic and complete.

a)

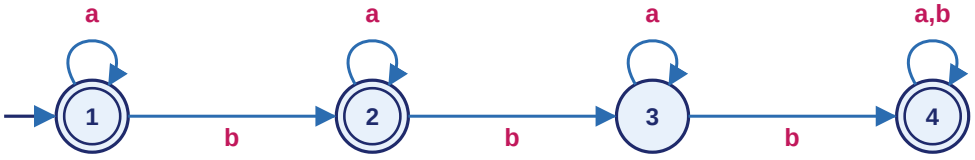


b) alphabet {a,b,c}

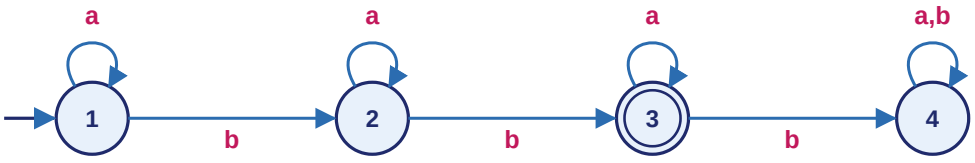


The loop label on state 2 is unreadable on the scan; a,c is the reading that makes the automaton consistent.

c)



d)



e) What is the link between c) and d)?

E8 — Minimise (Ch2)

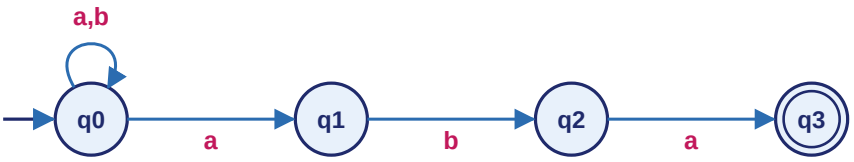
★ DS | slides 60–61 style | Cheat sheet: Minimisation

	a	b
→1	2	5
2	6	3
*3	6	5
4	2	7
5	6	1
6	2	7
*7	2	1

1. Remove the unreachable states.
2. Minimise (write every Step).
3. Draw the minimal automaton.
4. Give the language (words + RegEx).

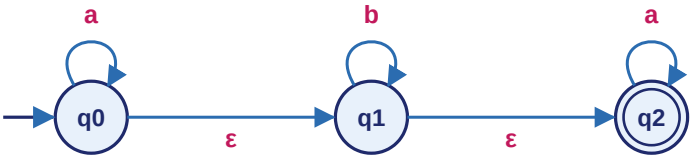
E9 — NFA → DFA, ε, complement (Ch3)

★ DS | slides 75–79 style | Cheat sheet: NFA→DFA, ε-closure



1. Why is this automaton an NFA? Give its transition table.
2. Convert it to a DFA. What is the language?
3. Give an automaton for the words that do **not** end with aba.

4. Same question 2 for this ε-NFA (compute the ε-closures first):



Solutions

E1

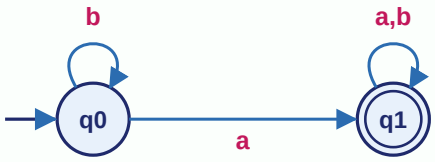
1. `\d{3}` = `ddd`
2. `\d*(0|5)` — her slide: (0-9)*(0|5). Any number ending with 0 or 5.
3. `4\d*7` (her slide: `4.*7`) — 47 ✔, 4 ✘, 417 ✔
4. `[1-9]\d{2}`

E2

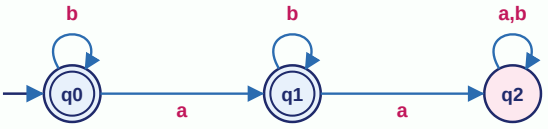
1. `(a*b)* = (a|b)*b` → **False**: ε ∈ (a*b)* but ε ∉ (a|b)*b (must end with b).
   `(ab)*a = a(ba)*` → **True**: sliding rule (rs)*r = r(sr)*. Both = {a, aba, ababa, ...}.
   `(a|b)* = (a*b*)*` → **True**: every word is a sequence of a's and b's, each letter is one a*b* block.
   `(ab)* = a*b*` → **False**: a ∈ a*b* but a ∉ (ab)* (also aab).
2. R = ends with bb, S = starts with b.
   In R not S: **abb** · in S not R: **b** · in both: **bb** · in neither: **a**

E3

1. At least one a: `b*a(a|b)*`

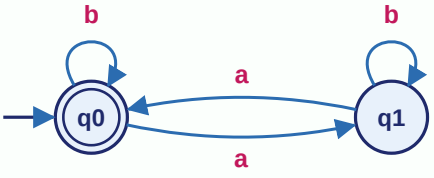


2. At most one a: `b* | b*ab*` = `b*(a|ε)b*`



q2 = dead state (a second a).

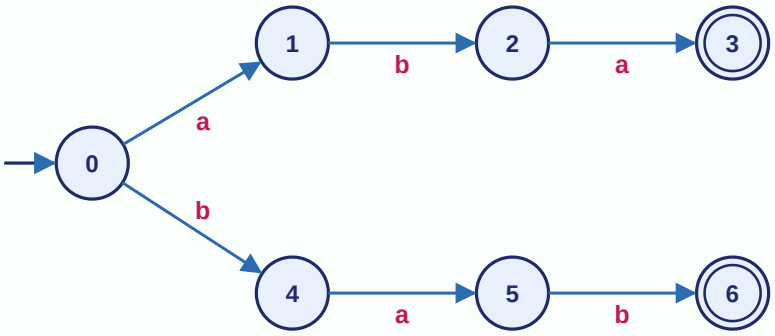
3. Even number of a: `b*(ab*ab*)*` = `(b|ab*a)*`



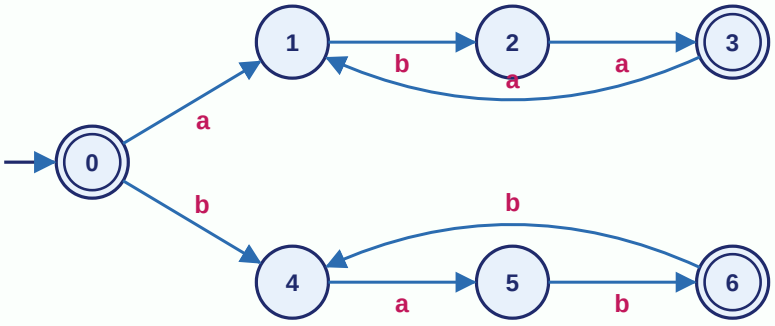
q0 = even, q1 = odd. ε is accepted (0 a's).

E4

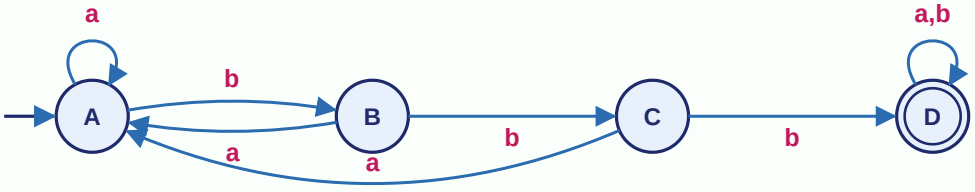
1. L = {aba, bab}: exactly two words. Glushkov: `a1 b2 a3 | b4 a5 b6`; First = {a1, b4}, Last = {a3, b6} ⇒ F = {3, 6}; successors a1→b2, b2→a3, b4→a5, a5→b6.



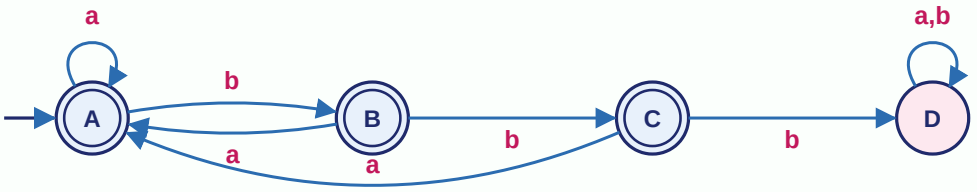
2. L = repetitions of aba only, or repetitions of bab only: {ε, aba, abaaba, ..., bab, babbab, ...}. No mixing: ababab (= aba·bab) ✘ — same idea as a*|b*.
Glushkov: `(a1 b2 a3)* | (b4 a5 b6)*`. Nullable ⇒ **0 ∈ F**. F = {0, 3, 6}. Successors: a1→b2, b2→a3, **a3→a1** (star), b4→a5, a5→b6, **b6→b4** (star).



3. Contains bbb: `(a|b)*bbb(a|b)*`. State = number of b's read in a row (A: 0, B: 1, C: 2, D: bbb found). An a sends back to A.



4. Complement of 3: the DFA is complete ⇒ **flip the states** (A, B, C final, D non-final = dead).

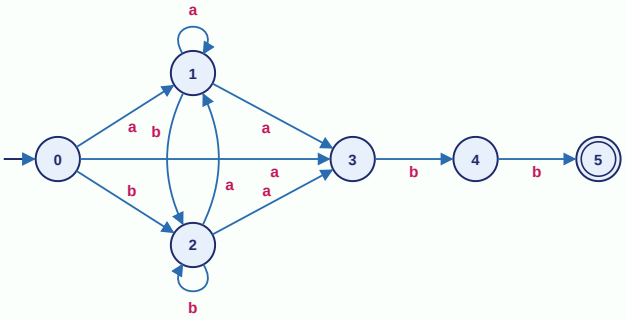


RegEx: `(a|ba|bba)*(ε|b|bb)` — exactly the class exercise of slide 61! Blocks a, ba, bba never make 3 b's; the end adds at most bb.

E5

1. `(a1 | b2)* a3 b4 b5` · First = {a1, b2, a3} · Last = {b5} · not nullable ⇒ **F = {5}**

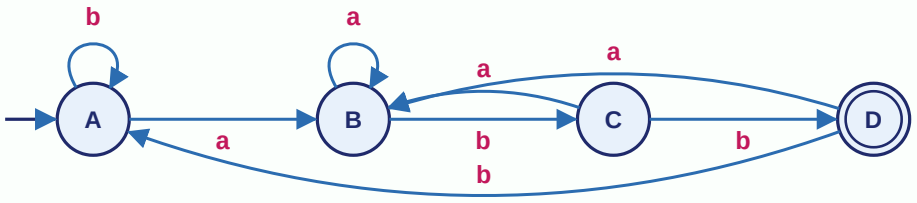
	successors
ϕ	a1, b2, a3
a1	a1, b2, a3
b2	a1, b2, a3
a3	b4
b4	b5
b5	ϕ



2. Not deterministic: from 0 (and 1, 2), reading a goes to **1 and 3** ⇒ ambiguity ⇒ NFA. Also 3, 4 have no a-transition.

3. Rows 0, 1, 2 are identical ⇒ call them S. Subset construction:

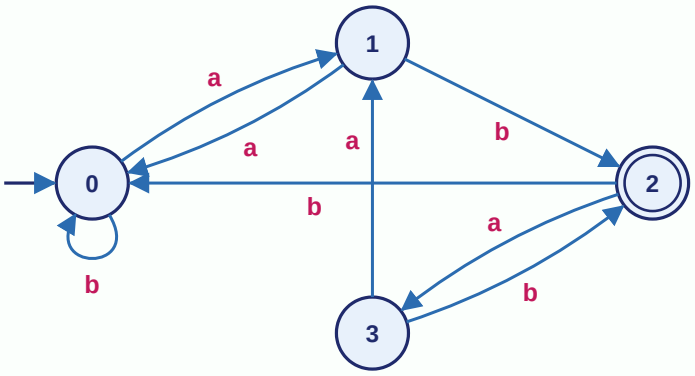
```
δ'([S],a)   = [S,3]  ← new        δ'([S],b)   = [S]
δ'([S,3],a) = [S,3]               δ'([S,3],b) = [S,4]  ← new
δ'([S,4],a) = [S,3]               δ'([S,4],b) = [S,5]  ← new
δ'([S,5],a) = [S,3]               δ'([S,5],b) = [S]
A = [S]   B = [S,3]   C = [S,4]   *D = [S,5]  (contains 5)
```



4. Step0: (D) (A,B,C) · Step1: C –b→ D final, A and B –b→ non-final ⇒ (D) (C) (A,B) · Step2: B –b→ C, A –b→ A ⇒ (D) (C) (B) (A). All separated ⇒ **minimal (4 states)**.

5. aabb: A –a→ B –a→ B –b→ C –b→ D ✔ accepted · abab: A –a→ B –b→ C –a→ B –b→ C ✘ (C not final).

E6



1. See drawing (0 loops on b).
2. Every state has exactly one a and one b ⇒ **complete** and **unambiguous** ⇒ **deterministic**.
3. ab: 0→1→2 ✔ · aab: 0→1→0→0 ✘ · abab: 0→1→2→3→2 ✔ · abb: 0→1→2→0 ✘
4. Step0: (2) (0,1,3) · Step1: 1 –b→ 2 and 3 –b→ 2 (final), 0 –b→ 0 ⇒ (2) (0) (1,3) · Step2: 1 –a→ 0, 3 –a→ 1: different groups ⇒ (2) (0) (1) (3) ⇒ already minimal.

E7

a) Words over {a,b} that contain **ba**: `(a|b)*ba(a|b)*` = `a*bb*a(a|b)*`. Deterministic and complete.

b) Words over {a,b,c} that **start with a and end with bc**: `a(a|b|c)*bc`. State 3 = last letter b, 4 = ends with bc. Deterministic, **not complete** (state 1 has no b, c ⇒ add a dead state).

c) Number of b **different from 2** (0, 1, or at least 3): `a* | a*ba* | a*ba*ba*b(a|b)*`. Deterministic and complete.

d) Exactly two b: `a*ba*ba*`. State 4 is a dead state. Deterministic and complete.

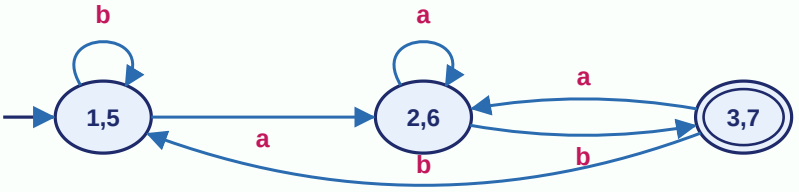
e) Same automaton, final and non-final states flipped ⇒ d) is the **complement** of c).

> Method for any automaton: find what each state "remembers" (number of b's, last letter...), then follow start → final paths: loop = *, choice = |.

E8

1. Nothing goes to 4 ⇒ 4 is unreachable ⇒ delete it.

```
Step0: (3,7) (1,2,5,6)
Step1: 2 —b→ 3 and 6 —b→ 7 (final); 1 —b→ 5, 5 —b→ 1 (non-final)
       ⇒ (3,7) (2,6) (1,5)
Step2: 1: a→2, b→5   5: a→6, b→1   ⇒ same groups
       2: a→6, b→3   6: a→2, b→7   ⇒ same groups
       3: a→6, b→5   7: a→2, b→1   ⇒ same groups
       stable ⇒ fusion: {1,5} {2,6} {3,7}
```



4. "1,5" = nothing useful, "2,6" = last letter a, "3,7" = ends with ab. Language: words ending with **ab**: `(a|b)*ab`.

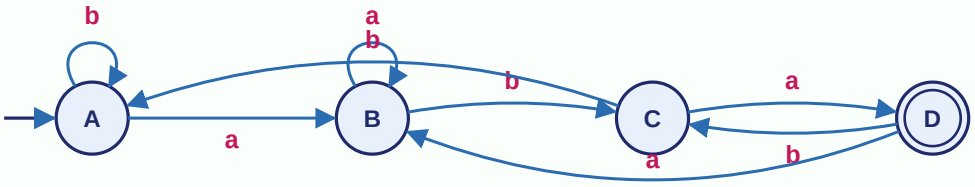
E9

1. From q0, reading a goes to q0 **and** q1 ⇒ ambiguous ⇒ NFA.

	a	b
→q0	{q0,q1}	q0
q1	-	q2
q2	q3	-
*q3	-	-

2.

```
δ'([q0],a) = [q0,q1] ← new
δ'([q0],b) = [q0]
δ'([q0,q1],a) = [q0,q1]
δ'([q0,q1],b) = [q0,q2] ← new
δ'([q0,q2],a) = [q0,q1,q3] ← new
δ'([q0,q2],b) = [q0]
δ'([q0,q1,q3],a) = [q0,q1]
δ'([q0,q1,q3],b) = [q0,q2]
A=[q0] B=[q0,q1] C=[q0,q2] *D=[q0,q1,q3]
```

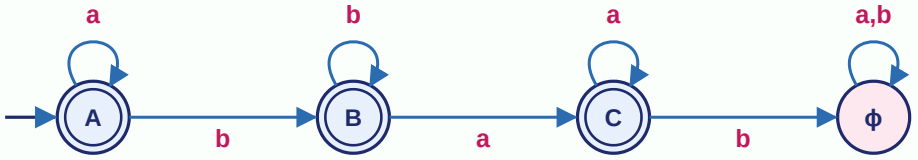


Language: words ending with **aba**: `(a|b)*aba`.

3. The DFA is complete ⇒ flip: A, B, C final, D non-final. (Never flip the NFA directly!)

4. ε-closure(q0) = {q0,q1,q2} · ε-closure(q1) = {q1,q2} · ε-closure(q2) = {q2}

```
A = {q0,q1,q2}* : a → {q0,q2} → closure {q0,q1,q2} = A
                  b → {q1}    → closure {q1,q2} = B*
B = {q1,q2}*    : a → {q2} = C*      b → {q1} → {q1,q2} = B
C = {q2}*       : a → C              b → ϕ (dead)
```



All of A, B, C contain q2 ⇒ final. Language: `a*b*a*`.